

Guide Pédagogique : Redux Classique avec React (Version Récente)

Introduction à Redux

Redux est une bibliothèque de gestion d'état prévisible pour les applications JavaScript. Il aide à écrire des applications qui se comportent de manière cohérente, s'exécutent dans différents environnements (client, serveur et natif), et sont faciles à tester. Bien que Redux Toolkit soit la méthode recommandée pour utiliser Redux aujourd'hui, ce guide se concentrera sur l'approche "classique" de Redux, sans l'utilisation de Redux Toolkit, pour une compréhension approfondie des concepts fondamentaux.

Les Trois Principes de Redux

1. **Source de vérité unique (Single Source of Truth)** : L'état global de votre application est stocké dans un seul objet à l'intérieur d'un seul *Store*. Cela simplifie le débogage et la compréhension de l'état de l'application.
2. **L'état est en lecture seule (State is Read-Only)** : La seule façon de modifier l'état est d'émettre une *Action*, un objet JavaScript décrivant ce qui s'est passé. Cela garantit que les vues et les callbacks réseau ne modifient jamais directement l'état.
3. **Les modifications sont faites avec des fonctions pures (Changes are Made with Pure Functions)** : Pour spécifier comment l'état est transformé par les actions, vous écrivez des *Reducers* purs. Les reducers prennent l'état précédent et une action, et retournent le nouvel état, sans effets secondaires.

Concepts Clés de Redux

- **Store** : L'objet qui contient l'état de l'application. Il n'y a qu'un seul store dans une application Redux. Il est créé avec `createStore`.
- **Actions** : Des objets JavaScript simples qui décrivent ce qui s'est passé. Ils doivent avoir une propriété `type` qui indique le type d'action effectuée. Elles sont envoyées au store via `store.dispatch(action)`.
- **Reducers** : Des fonctions pures qui prennent l'état actuel de l'application et une action, et retournent un nouvel état. Ils ne doivent jamais modifier l'état directement, mais plutôt retourner une nouvelle copie de l'état modifié.

- **Dispatch** : La méthode du store utilisée pour envoyer des actions. C'est le seul moyen de déclencher un changement d'état.
- **Souscription (Subscribe)** : La méthode du store qui permet d'enregistrer une fonction de rappel qui sera exécutée chaque fois que l'état du store change.

Intégration de Redux Classique avec React

Pour connecter Redux à une application React, nous utilisons la bibliothèque `react-redux`. Elle fournit des bindings pour React qui permettent aux composants de lire l'état du store Redux et de déclencher des actions.

Installation

```
npm install redux react-redux  
# ou  
yarn add redux react-redux
```

Provider

Le composant `<Provider>` de `react-redux` rend le store Redux disponible pour tous les composants imbriqués. Il doit envelopper le composant racine de votre application React.

```
// src/index.js ou src/main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import { createStore } from 'redux';
import { Provider } from 'react-redux';
import App from './App';
import rootReducer from './reducers'; // Nous allons créer ce fichier

const store = createStore(rootReducer);

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </React.StrictMode>
);
```

connect (pour les composants de classe ou fonctionnels sans hooks)

La fonction `connect` est une fonction d'ordre supérieur (Higher-Order Component - HOC) qui connecte un composant React au store Redux. Elle prend deux arguments optionnels :

`mapStateToProps` et `mapDispatchToProps` .

- `mapStateToProps(state, ownProps)` : Une fonction qui reçoit l'état du store Redux et les props du composant, et retourne un objet de props qui seront passées au composant. Elle est appelée chaque fois que l'état du store change.
- `mapDispatchToProps(dispatch, ownProps)` : Une fonction qui reçoit la fonction `dispatch` du store Redux et les props du composant, et retourne un objet de props qui seront des fonctions de rappel pour déclencher des actions.

```
// Exemple avec connect (pour information, nous utiliserons les hooks pour
import React from 'react';
import { connect } from 'react-redux';

const MyComponent = ({ myValue, increment }) => (
  <div>
    <p>Valeur : {myValue}</p>
    <button onClick={increment}>Incrémenter</button>
  </div>
);

const mapStateToProps = (state) => ({
  myValue: state.counter.value,
});

const mapDispatchToProps = (dispatch) => ({
  increment: () => dispatch({ type: 'INCREMENT' }),
});

export default connect(mapStateToProps, mapDispatchToProps)(MyComponent);
```

Hooks Redux (`useSelector` , `useDispatch`) (Recommandé pour React fonctionnel)

Avec React 16.8+, les hooks sont la manière recommandée d'interagir avec Redux dans les composants fonctionnels. Ils sont plus simples et plus modernes que `connect` .

- `useSelector(selectorFunction)` : Un hook qui permet d'extraire des données de l'état du store Redux. Il prend une fonction sélecteur comme argument, qui reçoit l'état global et retourne la partie de l'état dont le composant a besoin.
- `useDispatch()` : Un hook qui retourne une référence à la fonction `dispatch` du store Redux. Vous pouvez l'utiliser pour envoyer des actions.

```
// Exemple avec hooks (sera utilisé pour le TP)
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';

const Counter = () => {
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div>
      <p>Compteur : {count}</p>
      <button onClick={() => dispatch({ type: 'INCREMENT' })}>Incrémenter</button>
      <button onClick={() => dispatch({ type: 'DECREMENT' })}>Décrémenter</button>
    </div>
  );
};

export default Counter;
```

TP Simple : Un Compteur Redux Classique avec React

Nous allons créer une application React simple avec un compteur qui utilise Redux classique pour gérer son état.

Étape 1 : Initialisation du projet React

Créez un nouveau projet React. Nous utiliserons Vite pour une configuration rapide.

```
npm create vite my-redux-counter -- --template react
cd my-redux-counter
npm install
```

Étape 2 : Installation des dépendances Redux

```
npm install redux react-redux
```

Étape 3 : Création des Actions

Les actions sont des objets qui décrivent ce qui s'est passé. Pour notre compteur, nous aurons deux actions : `INCREMENT` et `DECREMENT`.

Créez un fichier `src/actions/counterActions.js` :

```
// src/actions/counterActions.js
export const INCREMENT = 'INCREMENT';
export const DECREMENT = 'DECREMENT';

export const increment = () => ({
  type: INCREMENT,
});

export const decrement = () => ({
  type: DECREMENT,
});
```

Étape 4 : Création du Reducer

Le reducer est une fonction pure qui prend l'état actuel et une action, et retourne le nouvel état. Il gérera la logique de mise à jour de notre compteur.

Créez un fichier `src/reducers/counterReducer.js` :

```
// src/reducers/counterReducer.js
import { INCREMENT, DECREMENT } from '../actions/counterActions';

const initialState = {
  value: 0,
};

const counterReducer = (state = initialState, action) => {
  switch (action.type) {
    case INCREMENT:
      return { ...state, value: state.value + 1 };
    case DECREMENT:
      return { ...state, value: state.value - 1 };
    default:
      return state;
  }
};

export default counterReducer;
```

Étape 5 : Combinaison des Reducers (si plusieurs)

Pour une application plus grande, vous auriez plusieurs reducers. `combineReducers` de Redux permet de les combiner en un seul reducer racine. Pour notre TP, nous n'avons qu'un seul reducer, mais il est bon de montrer comment le structurer.

Créez un fichier `src/reducers/index.js` :

```
// src/reducers/index.js
import { combineReducers } from 'redux';
import counterReducer from './counterReducer';

const rootReducer = combineReducers({
  counter: counterReducer, // 'counter' sera la clé dans l'état global pour
});

export default rootReducer;
```

Étape 6 : Création du Store Redux

Le store est l'endroit où l'état de l'application est stocké. Nous le créerons en utilisant `createStore` et notre `rootReducer`.

Modifiez `src/main.jsx` (ou `src/index.js` si vous utilisez Create React App) :

```
// src/main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import { createStore } from 'redux';
import { Provider } from 'react-redux';
import App from './App.jsx'; // Assurez-vous que le chemin est correct
import rootReducer from './reducers';
import './index.css'; // Pour les styles par défaut de Vite

const store = createStore(rootReducer);

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </React.StrictMode>,
);
```

Étape 7 : Création du Composant React Counter

Nous allons créer un composant fonctionnel React qui affichera la valeur du compteur et permettra d'incrémenter/décrémenter via des boutons, en utilisant les hooks `useSelector` et `useDispatch`.

Créez un fichier `src/components/Counter.jsx` :


```
// src/components/Counter.jsx
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement } from '../actions/counterActions';

const Counter = () => {
  // Extrait la valeur du compteur de l'état Redux
  const count = useSelector(state => state.counter.value);
  // Obtient la fonction dispatch pour envoyer des actions
  const dispatch = useDispatch();

  return (
    <div style={{ textAlign: 'center', marginTop: '50px' }}>
      <h1>Compteur Redux Classique</h1>
      <p style={{ fontSize: '2em', margin: '20px 0' }}>Valeur : {count}</p>
      <button
        onClick={() => dispatch(increment())}
        style={{
          padding: '10px 20px',
          fontSize: '1em',
          margin: '0 10px',
          cursor: 'pointer',
        }}
      >
        Incrémenter
      </button>
      <button
        onClick={() => dispatch(decrement())}
        style={{
          padding: '10px 20px',
          fontSize: '1em',
          margin: '0 10px',
          cursor: 'pointer',
        }}
      >
        Décrémenter
      </button>
    </div>
  );
};

export default Counter;
```

Étape 8 : Intégration du Composant Counter dans App.jsx

Modifiez `src/App.jsx` pour afficher notre composant `Counter`.

```
// src/App.jsx
import React from 'react';
import Counter from '../components/Counter';
import './App.css'; // Pour les styles par défaut de Vite

function App() {
  return (
    <div className="App">
      <Counter />
    </div>
  );
}

export default App;
```

Étape 9 : Lancement de l'application

```
npm run dev
```

Ouvrez votre navigateur à l'adresse indiquée (généralement `http://localhost:5173/`) et vous devriez voir votre application de compteur Redux fonctionner.

Conclusion

Ce guide a couvert les fondamentaux de Redux classique et son intégration avec React en utilisant les hooks `useSelector` et `useDispatch`. Bien que Redux Toolkit soit la méthode moderne et recommandée pour la plupart des applications, comprendre les concepts sous-jacents de Redux classique est essentiel pour une base solide. Ce TP simple vous a permis de mettre en pratique ces concepts et de voir comment l'état est géré de manière prévisible dans une application React.

Références

- [1] [Redux Official Documentation](#)
- [2] [React Redux Official Documentation](#)
- [3] [Migrating to Modern Redux - Redux Documentation](#)
- [4] [Releases · reduxjs/react-redux - GitHub](#)
- [5] [Just Redux: The Complete Guide - DEV Community](#)
- [6] [A quick guide to integrating React-Redux with React app\(Legacy way\) - Medium](#)
- [7] [Hooks | React Redux](#)
- [8] [Introduction on React-Redux using Hooks \(useSelector & useDispatch\) - Medium](#)
- [9] [How to Use useSelector and useDispatch React-Redux - BuiltIn](#)